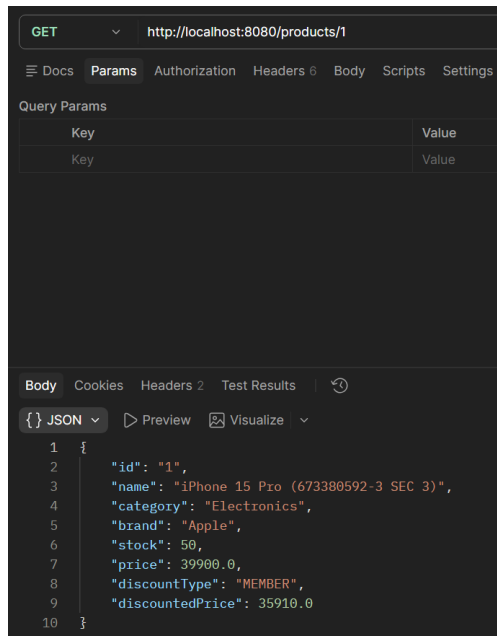


Lab10_Webclient

Screenshot ทดสอบ endpoint

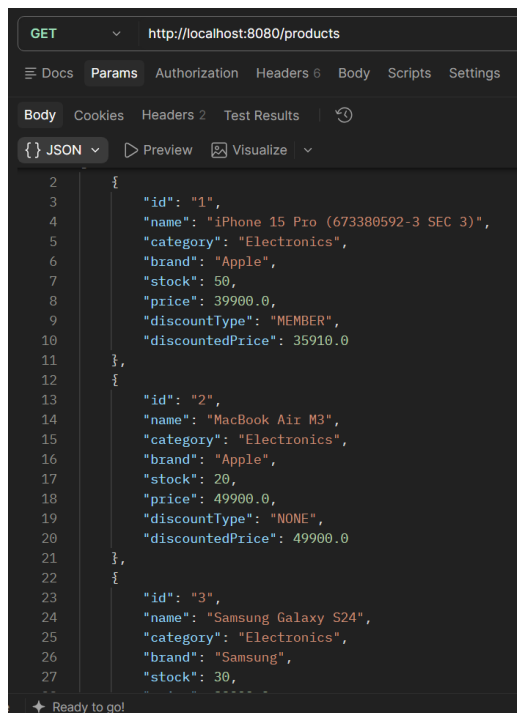
1. GET /products/{id}

ทดสอบ: GET <http://localhost:8080/products/1>



2. GET /products

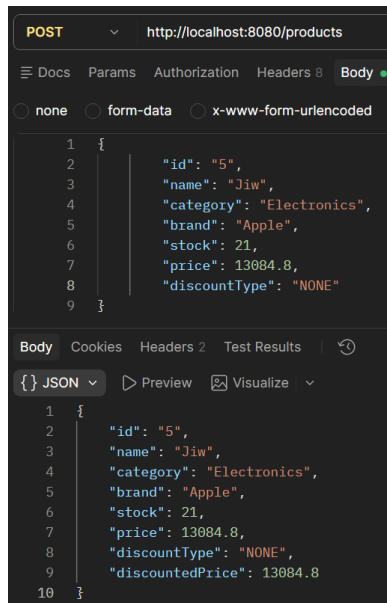
ทดสอบ: GET <http://localhost:8080/products>



3. POST /products

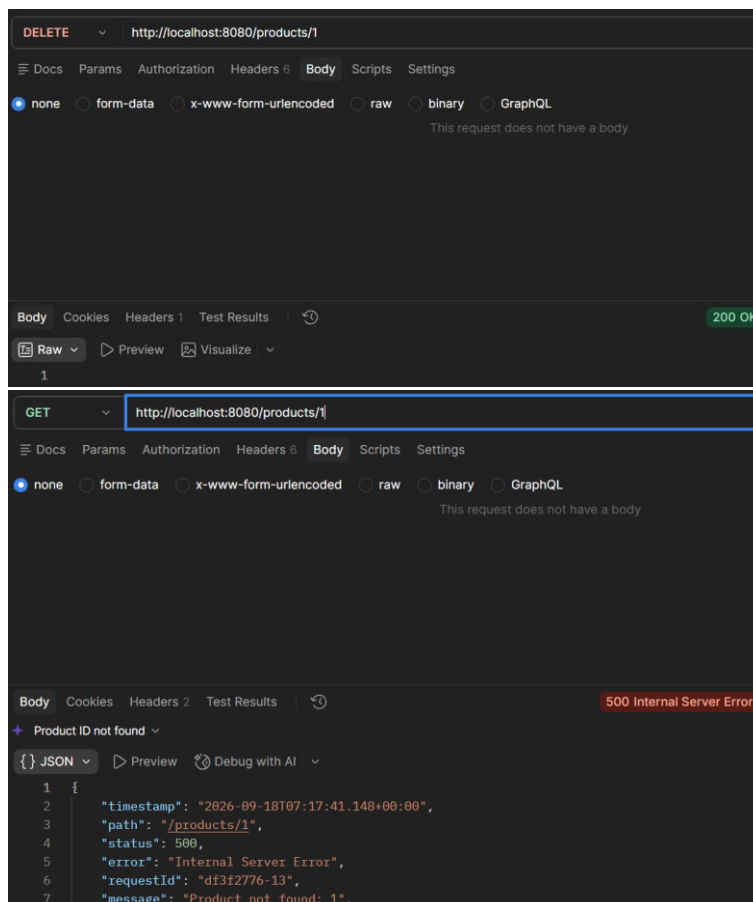
ทดสอบ: POST <http://localhost:8080/products>

Body: { "name": "Pawarisa", "price": 999.0, ... }



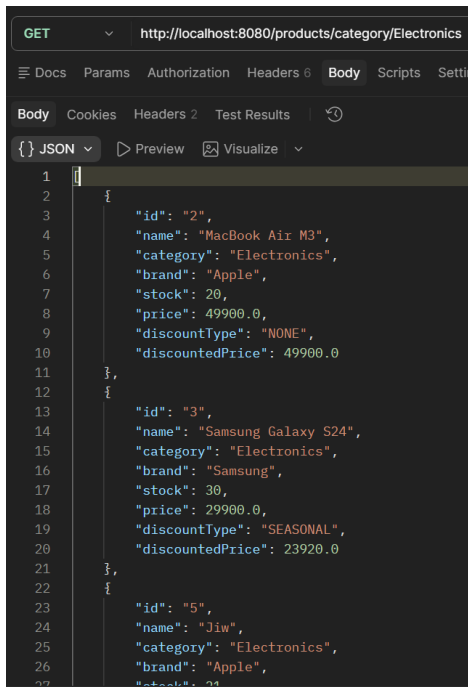
4. DELETE /products/{id}

ทดสอบ: DELETE <http://localhost:8080/products/1>



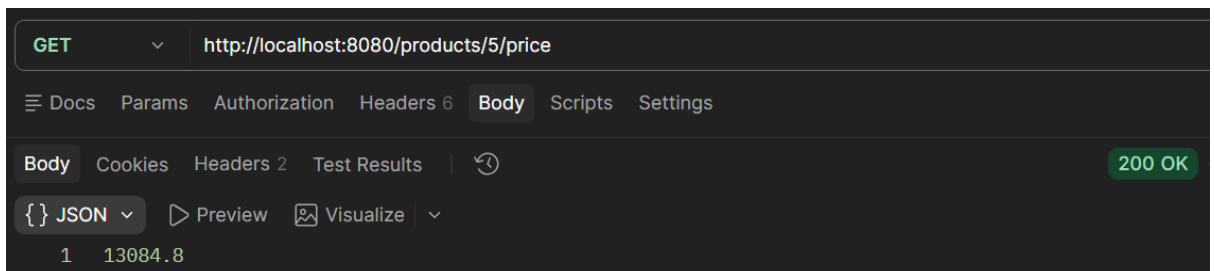
5. GET /products/category/{category}

ทดสอบ: GET <http://localhost:8080/products/category/Electronics>



6. GET /products/{id}/price

ทดสอบ: GET <http://localhost:8080/products/1/price>



คำถาม Lab 10

1. Reactive Programming vs Blocking — ต่างกันอย่างไร

ตอบ Reactive Programming ทำงานแบบไม่ต้องรอผลลัพธ์ ทำงานอื่นต่อได้ Blocking จะต้องรอให้การทำงานแรกเสร็จก่อน ถึงจะทำงานต่อไปได้

2. Mono vs Flux — ใช้กรณีไหน และทำไม

ตอบ Mono ใช้เมื่อคาดว่าจะมีข้อมูล 0 หรือ 1 ค่า

Flux ใช้เมื่อคาดว่าจะมีข้อมูล 0 ถึงหลายค่า

ดังนั้นเลือกใช้ตามจำนวนข้อมูลที่ต้องการรับกลับมา

3. WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

ตอบ .get() ใช้กำหนดว่าจะส่ง HTTP Request แบบ GET

.uri() ใช้กำหนด URL หรือ endpoint ที่ต้องการเรียก

.retrieve() ใช้ส่ง Request และรับ Response กลับมา

.bodyToMono() ใช้แปลง Response Body เป็นข้อมูลแบบ Mono เช่น Product 1 รายการ

.bodyToFlux() ใช้แปลง Response Body เป็นข้อมูลแบบ Flux เช่น Product หลายรายการ

4. Code ของ Repository, Service, Controller พร้อม comment อธิบาย

ตอบ

Code ของ Repository

```
public class ProductRepository {

    // ใช้เก็บข้อมูล Product ไว้ในหน่วยความจำ
    // โดยใช้ id เป็น key
    private final Map<String, Product> store = new ConcurrentHashMap<>();

    // Constructor สำหรับเพิ่มข้อมูล Product ตัวอย่าง
    public ProductRepository() {
        store.put("1", new Product("1", "iPhone 15 Pro (673380592-3 SEC 3)",
            "Electronics", "Apple", 50, 39900.0, "MEMBER"));

        store.put("2", new Product("2", "MacBook Air M3",
            "Electronics", "Apple", 20, 49900.0, "NONE"));

        store.put("3", new Product("3", "Samsung Galaxy S24",
            "Electronics", "Samsung", 30, 29900.0, "SEASONAL"));
    }

    // ค้นหา Product 1 รายการจาก id
    // ถ้าไม่พบจะคืน Mono.empty()
    public Mono<Product> findById(String id) {
        Product product = store.get(id);

        if (store.get(id) == null) {
            return Mono.empty();
        } else {
```

```

        return Mono.just(product);
    }
}

// ค้นหา Product ทั้งหมด
// แปลงข้อมูลจาก Collection เป็น Flux
public Flux<Product> findAll() {
    return Flux.fromIterable(store.values());
}

// บันทึก Product ลงใน store
// แล้วคืน Product กลับมาในรูปแบบ Mono
public Mono<Product> save(Product product) {
    store.put(product.getId(), product);
    return Mono.just(product);
}

// ลบ Product ตาม id
// แล้วคืน Mono<Void>
public Mono<Void> deleteById(String id) {
    store.remove(id);
    return Mono.empty();
}

// ค้นหา Product ตาม category
// โดยไม่สนใจตัวพิมพ์เล็กหรือตัวพิมพ์ใหญ่
public Flux<Product> findByCategory(String category) {
    return findAll()
        .filter(p -> p.getCategory().equalsIgnoreCase(category));
}
}

```

Code ของ Service

```

@Service
public class ProductService {

    // เชื่อมต่อกับ ProductRepository
    private final ProductRepository repository;
}

```

```
// Constructor Injection
public ProductService(ProductRepository repository) {
    this.repository = repository;
}

// ค้นหา Product 1 รายการจาก id
// ถ้าไม่พบให้แสดงข้อความ Product not found
public Mono<Product> getByld(String id) {
    return repository.findByld(id)
        .switchIfEmpty(
            Mono.error(
                new RuntimeException("Product not found: " + id)
            )
        );
}

// ค้นหา Product ทั้งหมด
public Flux<Product> getAll() {
    return repository.findAll();
}

// บันทึก Product
// ถ้า Product ยังไม่มี id ให้สร้าง id ใหม่ด้วย UUID
public Mono<Product> save(Product product) {
    if (product.getId() == null) {
        product.setId(UUID.randomUUID().toString());
    }

    return repository.save(product);
}

// ลบ Product ตาม id
public Mono<Void> delete(String id) {
    return repository.deleteByld(id);
}

// ค้นหา Product ตาม category
```

```

public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

// คำนวณราคาหลังส่วนลดของ Product
public Mono<Double> getDiscountedPrice(String id) {
    return getById(id)
        .map(p -> p.getDiscountedPrice());
}
}

```

Code ของ Controller

```

@RestController
@RequestMapping("/products")
public class ProductController {

    // เชื่อมต่อกับ ProductService
    private final ProductService service;

    // Constructor Injection
    public ProductController(ProductService service) {
        this.service = service;
    }

    // GET /products/{id}
    // ค้นหา Product 1 รายการจาก id
    public Mono<Product> getById(@PathVariable String id) {
        return service.getById(id);
    }

    // GET /products
    // ค้นหา Product ทั้งหมด
    @GetMapping
    public Flux<Product> getAll() {
        return service.getAll();
    }

    // POST /products

```

```
// รับ Product จาก Request Body แล้วบันทึก
@PostMapping
public Mono<Product> save(@RequestBody Product product) {
    return service.save(product);
}

// DELETE /products/{id}
// ลบ Product ตาม id
@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
    return service.delete(id);
}

// GET /products/category/{category}
// ค้นหา Product ตาม category
@GetMapping("/category/{category}")
public Flux<Product> getByCategory(@PathVariable String category) {
    return service.getByCategory(category);
}

// GET /products/{id}/price
// คำนวณและคืนราคาหลังส่วนลด
@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(@PathVariable String id) {
    return service.getDiscountedPrice(id);
}
}
```